**Task 1: Guided Activity, Create and break a retain cycle**

```
import Foundation
class Person {
    let name: String
    var apartment: Apartment?
    init(name: String) {
        self.name = name
    }
    deinit {
        print("Person \(name) deallocated")
    }
}
class Apartment {
    let unit: String
    var tenant: Person?
    init(unit: String) {
        self.unit = unit
    }
    deinit {
        print("Apartment \(unit) deallocated")
    }
}

var person: Person? = Person(name: "Anu")
var apartment: Apartment? = Apartment(unit: "101")
person!.apartment = apartment
apartment!.tenant = person
person = nil
apartment = nil
print("Both variables set to nil")
```

**OUTPUT:**

```
Both variables set to nil
```

```
import Foundation
class Person {
    let name: String
    var apartment: Apartment?
    init(name: String) {
        self.name = name
    }
    deinit {
        print("Person \(name) deallocated")
    }
```

```
}
class Apartment {
    let unit: String
    weak var tenant: Person?
    init(unit: String) {
        self.unit = unit
    }
    deinit {
        print("Apartment \(unit) deallocated")
    }
}

var person: Person? = Person(name: "Anu")
var apartment: Apartment? = Apartment(unit: "101")
person!.apartment = apartment
apartment!.tenant = person
person = nil
apartment = nil
print("Both variables set to nil")
```

```
Person Anu deallocated
Apartment 101 deallocated
Both variables set to nil
```

**Explanation:**

Changing tenant to weak breaks the strong reference cycle, allowing both objects' reference counts to reach zero so their deinit methods can run.

**Task 2: Practice Task, Closure capture cycle**

```
import Foundation
class Downloader {
    let name: String
    var onFinish: (() -> Void)?
    init(name: String) {
        self.name = name
    }
    func start() {
        onFinish = {
            print("Finished \(self.name)")
        }
    }
    deinit {
        print("Downloader \(name) deallocated")
```

```
    }
}
var downloader: Downloader? = Downloader(name: "File1")
downloader?.start()
downloader = nil
print("Downloader variable set to nil")
```

**OUTPUT:**

```
Downloader variable set to nil
```

```
import Foundation
class Downloader {
    let name: String
    var onFinish: (() -> Void)?

    init(name: String) {
        self.name = name
    }
    func start() {
        onFinish = { [weak self] in
            guard let self = self else {
                return
            }
            print("Finished \(self.name)")
        }
    }
    deinit {
        print("Downloader \(name) deallocated")
    }
}
var downloader: Downloader? = Downloader(name: "File1")
downloader?.start()
downloader?.onFinish?()
downloader = nil
print("Downloader variable set to nil")
```

```
Finished File1
Downloader File1 deallocated
Downloader variable set to nil
```

**Task 3: Predict Task, When does the object die?**

```
class Box {
    deinit {
        print("Box gone")
    }
}
var a: Box? = Box()
var b = a
var c = a
a = nil
b = nil
print("before c")
c = nil
print("end")
```

**OUTPUT:**

```
before c
Box gone
end
```

```
class Box {
    deinit {
        print("Box gone")
    }
}

var a: Box? = Box()
var b = a
var c = a

a = nil
b = nil

print("before c")

c = nil

print("end")
```

Expected output:

```
before c
Box gone
end
```

## 2. Why is this the output?

1. `a`, `b`, and `c` all hold references to the same `Box` object.
2. `a = nil` removes one reference, but `b` and `c` still retain the object.
3. `b = nil` removes another reference, but `c` still retains it.
4. `print("before c")` executes while the object is still alive.
5. `c = nil` removes the last strong reference, so `deinit` prints `Box gone`.
6. Finally, `print("end")` executes.

### 3. When to use strong, weak, and unowned

| Reference type | When to use | Keeps object alive? | Can be nil? |
|---|---|---|---|
| strong (default) | When the object should stay alive while referenced | Yes | Depends on declaration |
| weak | When the referenced object may be deallocated first, such as a delegate | No | Yes; automatically becomes nil |
| unowned | When the referenced object is guaranteed to outlive the reference | No | No, for a standard non-optional unowned reference |